

Beyond Accuracy: Characterizing Code Comprehension Capabilities in (Large) Language Models

Felix Mächtle
ITS, University of Luebeck
Luebeck, Germany

Jan-Niclas Serr
ITS, University of Luebeck
Luebeck, Germany

Nils Loose
ITS, University of Luebeck
Luebeck, Germany

Thomas Eisenbarth
ITS, University of Luebeck
Luebeck, Germany

Abstract

Large Language Models (LLMs) are increasingly integrated into software engineering workflows, yet current benchmarks provide only coarse performance summaries that obscure the diverse capabilities and limitations of these models. This paper investigates whether LLMs' code-comprehension performance aligns with traditional human-centric software metrics or instead reflects distinct, non-human regularities. We introduce a diagnostic framework that reframes code understanding as a binary input-output consistency task, enabling the evaluation of classification and generative models. Using a large-scale dataset, we correlate model performance with traditional, human-centric complexity metrics, such as lexical size, control-flow complexity, and abstract syntax tree structure. Our analyses reveal minimal correlation between human-defined metrics and LLM success (AUROC 0.63), while shadow models achieve substantially higher predictive performance (AUROC 0.86), capturing complex, partially predictable patterns beyond traditional software measures. These findings suggest that LLM comprehension reflects model-specific regularities only partially accessible through either human-designed or learned features, emphasizing the need for benchmark methodologies that move beyond aggregate accuracy and toward instance-level diagnostics, while acknowledging fundamental limits in predicting correct outcomes.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Software and its engineering**;

Keywords

Code Comprehension, Model Evaluation and Benchmarking, Machine Learning for Software Engineering

1 Introduction

Large Language Models (LLMs) have profoundly reshaped software development, facilitating tasks from automated code generation over debugging support to vulnerability classification. As LLMs become integral to developers' workflows, accurately assessing their strengths and weaknesses across various coding tasks has grown essential. Typically, developers depend on benchmark results to guide model selection, viewing aggregate success metrics as reliable indicators of effectiveness. However, these aggregated metrics often conceal critical nuances in model performance.

Consider two software developers: Alice, who predominantly tackles complex algorithmic challenges requiring mathematical reasoning, and Bob, who primarily addresses web development. Both are interested in leveraging LLMs for productivity enhancement. When consulting standard benchmarks, such as SWE-bench [4],

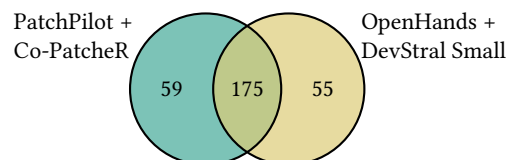


Figure 1: Overlap of solved benchmark tasks on the verified SWE-bench dataset by the top two open-source models. While overall performance differs minimally (just 4 unique tasks difference), their solved task types substantially diverge, demonstrating differences in model specialization.

they find performance metrics that aggregate success across diverse task sets. However, such generalized metrics provide limited guidance, obscuring which tasks or domains each model excels in.

Figure 1 illustrates this issue, showing the overlap in solved tasks between the two best-performing open-source models on the verified SWE-bench dataset (commit ID 40c982d). Although both models solve a similar total number of tasks, i.e., 234 tasks solved by model A (*OpenHands + DevStral Small*) and 230 tasks by model B (*PatchPilot + Co-PatcheR*), their strengths diverge. While 175 tasks overlap, model A uniquely solves 59 tasks and model B uniquely solves 55. This scenario implies that Alice might prefer model A if her tasks align closely with the unique strengths of that model, while Bob might favor model B due to its advantages.

Motivated by this practical gap, our work investigates the foundations of LLM code comprehension: do models succeed or fail due to human-interpretable code properties, or do they follow distinct, non-human regularities? Specifically, we examine whether model performance correlates with traditional human-centric software metrics (e.g., line count, variable number, cyclomatic complexity) or whether it reflects different, latent regularities inaccessible to human interpretation. To this end, we design a diagnostic framework that reframes code understanding as a binary input-output (I/O) consistency task, enabling the evaluation of classification and generative models. Using a large-scale dataset derived from CodeNet [8], we compare the predictive power of two approaches, i.e., (i) a classifier trained on human-engineered static code features, and (ii) a feature-free shadow model trained directly on raw code and I/O pairs.

Our analysis reveals that traditional software metrics correlate only weakly with model success (AUROC 0.63), suggesting that LLMs' comprehension capabilities diverge from human reasoning about code. In contrast, shadow models achieve moderately higher

predictive performance (AUROC 0.86), indicating that learned representations capture systematic patterns beyond handcrafted features, yet substantial prediction uncertainty remains. These findings highlight the need for new benchmarking methodologies that move beyond aggregate accuracy toward instance-level diagnostics, while acknowledging fundamental limits: LLM code comprehension reflects model-specific, partially predictable regularities that differ from human cognitive processes and resist complete characterization. All hyperparameters and implementation details are available in our GitHub Repository: <https://github.com/UzL-ITS/code-comprehension-capabilities-llms>.

2 Preliminaries

2.1 CodeNet

CodeNet [8] is an extensive dataset specifically created for research on code-related tasks. It consists of approximately 14 million code samples spanning over 50 programming languages, collected from popular online programming platforms.

2.2 (Large) Language Models

Early code classification models, such as CNNs, RNNs, and graph-based networks, captured local or sequential code patterns but lacked long-range contextual understanding. Transformer-based encoders like UniXcoder [3] addressed this through self-attention, enabling richer code representations across languages and tasks. More recently, decoder-only LLMs (e.g., GPT-4, CodeLlama, Mistral) have demonstrated strong generalization through large-scale pretraining and in-context reasoning.

2.3 SAGE

Shapley Additive Global importance (SAGE) [1] quantifies how much each feature contributes to a model’s predictive performance. Inspired by cooperative game theory, SAGE computes the expected reduction in loss when revealing a feature, averaged over all possible feature subsets. Thus, SAGE provides a global, dependence-aware measure of feature importance that accounts for correlations between inputs.

3 Methodology

Our goal is to test whether a model’s ability to understand code aligns with human-centric software metrics or instead reflect distinct, non-human regularities. To this end, we cast code comprehension as a binary classification problem with ground-truth labels (Section 3.1), distill a large set of static, execution-free metrics/features that summarize programs at the source and bytecode levels (Section 3.2), and present two predictors of LLM success: a classifier trained on human metrics (Section 3.3) and a feature-free shadow model trained directly on raw inputs (Section 3.4).

3.1 Task formulation and label construction

To evaluate the code comprehension capabilities of language models, an appropriate task formulation is required. Prior work, such as Gu *et al.* [2], assesses this by providing programs with concrete

inputs and verifying whether the model generates the correct output. However, since language models are also frequently applied to classification problems, we frame our evaluation as a classification.

Let p denote a program (source code), x an input, and y a candidate output. We define the ground-truth label

$$t(p, x, y) = [y = f_p(x)],$$

where f_p is the (deterministic) input–output mapping implemented by p . Positive examples (p, x, y) are obtained by executing p on x and recording the resulting output. To generate matched negative examples without changing the program or input distribution, we pair each (p, x) with an output y' drawn from a different input for the same program ($y' = f_p(x')$ with $x' \neq x \wedge y' \neq y$), yielding (p, x, y') . This in-program shuffling preserves lexical, stylistic, and domain characteristics of p while producing semantically incorrect I/O pairs.

Given a model M , we query M with (p, x, y) and obtain a binary judgment $\hat{t}_M(p, x, y) \in \{0, 1\}$ (*does y match the output of p on x ?*). We say that M succeeds on the sample if its judgment matches the ground truth:

$$s_M(p, x, y) = [\hat{t}_M(p, x, y) = t(p, x, y)].$$

All downstream analyses predict or explain s_M .

3.2 Human-centric metrics

To probe alignment with traditional software measures, we compute a feature vector $\phi(p, x, y)$ for each triple (p, x, y) without executing the program. Features fall into four families:

- **Size / lexical.** Character and token counts, identifier and comment counts, simple name statistics, and lengths of x and y (e.g., `code_chars`, `num_identifiers`, `len_input`).
- **Opcodes statistics.** Statistics of Python bytecode opcode IDs and names extracted from the compiled form of p (e.g., number of opcodes, per-opcode frequencies, entropy).
- **AST / graph structure.** Metrics computed on the abstract syntax tree of p and its derived graphs (e.g., node/edge counts, density, diameter, average shortest path length).
- **Control-flow / complexity.** Cyclomatic complexity, loop and branch counts, and basic-block statistics.

All features are obtained via static analysis of source or compiled bytecode. The full list of about 300 features can be seen in our GitHub Repository.

3.3 Predicting LLM success from human metrics

We train a supervised classifier g_θ on $(\phi(p, x, y), s_M)$, i.e., the human metrics for a sample, together with information on whether the model was successful for that sample. Thus, we assess how much human-centric metrics predict the success of a model.

To obtain global, dependence-aware feature importances, i.e., which metrics lead to model success/failure, we compute feature importance based on SAGE [1]. SAGE values are used to (i) rank features and (ii) construct compact models by retaining the smallest subset whose positive SAGE contributions account for 95% of the total positive mass. Comparing full vs. pruned models reveals whether a small set of human metrics suffices to predict s_M .

3.4 Shadow model (feature-free predictor)

Human-designed features may miss high-dimensional cues that influence LLM behavior. We therefore train a shadow model $\tilde{g}_\psi$ that predicts s_M directly from raw inputs. Concretely, we fine-tune an encoder for code (UniXcoder [3]) on serialized sequences

$$\langle p \text{ [SEP]} x \text{ [SEP]} y \rangle,$$

omitting handcrafted features. This model estimates whether a target model will answer correctly for a given (p, x, y) based solely on the code and input/output-context, serving as feature-free predictor.

3.5 Dataset construction

We focus on the Python portion of CodeNet [8]. For each program p , we synthesize inputs with a lightweight, type-aware fuzzer and execute p to collect outputs, producing 799,365 triples (p, x, y) . To prevent redundant or contradictory training signals, we collapse all program-input pairs that exhibit identical observable behavior for a given sample. Finally, the dataset is partitioned by problem statement: every tenth problem is assigned to the evaluation set, while the remaining problems form the training pool.

To respect the context limits of models and ensure stable preprocessing, we apply length filters of at most 5,000 characters for code and 500 characters for each of input and output. Before filtering, 152,803 pairs are assigned to the training split and 12,584 to the test split. After filtering, the final corpora contain 148,243 training and 12,563 evaluation pairs. All static features in Section 3.2 are computed from these filtered sources.

4 Experiments

4.1 Model Performance

Following prior benchmarks [2, 4], we begin by evaluating the performance of all models on our task. For models requiring training, we reuse the set of models selected in OCEAN [7], along with their publicly available implementations and training routines.

Experiment setup. We evaluate general-purpose LLMs and classic text baselines on the binary classification task described in Section 3.1. Models that require training are fit on the training split of our dataset, frozen API models are prompted to decide whether the output matches the program’s behavior on the given input. The prompt follows the style of CruXEval [2], adapted to our classification format. We report performance on the held-out test split. Results are shown in Table 1.

Discussion. We observe that model scale and pretraining quality play decisive roles: larger decoder-based models consistently outperform shallow text baselines. The largest model with reasoning capabilities, GPT-OSS 120B, represents the strongest system in our comparison. Interestingly, the much smaller yet code-specialized UniXcoder ranks second, demonstrating that compact models trained on domain-specific data can achieve competitive results. Moreover, the dominance of Transformer-based architectures across all high-performing systems highlights their effectiveness in capturing the structural and semantic dependencies inherent in code. The remainder of the paper focuses on the subset of working models, i.e., those achieving an F1 score above 0.65.

Table 1: Aggregated performance of different models on our benchmark dataset. “I.” denotes instruction-tuned variants.

Model	Accuracy	Precision	Recall	F1
CodeLlama 13B I.	0.506	0.033	0.600	0.062
Llama 3.3 70B I.	0.738	0.514	0.931	0.662
Mistral Small 24B I.	0.744	0.556	0.892	0.685
GPT-OSS 120B	0.960	0.926	0.995	0.959
Phi-4	0.733	0.554	0.862	0.674
CNN	0.500	0.500	0.728	0.593
FNN + N-grams	0.539	0.551	0.417	0.475
FNN + TF-IDF	0.511	0.533	0.179	0.268
LSTM	0.500	0.500	0.000	0.000
UniXcoder	0.815	0.790	0.860	0.823

4.2 Human Metrics

Having established per-sample success labels for each model, we investigate whether traditional human-designed code properties correlate with LLM performance. We train a supervised classifier on the static features described in Section 3.2 to predict model success, then apply SAGE [1] to identify which metrics matter most.

Experiment Setup. We train an XGBoost classifier on an 80/20 stratified train/test split, with features $\phi(p, x, y)$ as input and the binary success label s_M as target. To obtain global feature importance rankings, we compute SAGE values on the held-out split. Following the 95% coverage criterion from Section 3.3, we retain the smallest feature subset whose positive SAGE contributions account for 95% of total positive mass. We retrain XGBoost on this reduced set and report test-set AUROC for both full and pruned models.

Results. Across all models, SAGE pruning retained only 16.8 features on average (5.8%). Despite this drastic reduction, pruned models achieved mean AUROC of 0.634 versus 0.554 for full-feature baselines, an improvement of +0.080. The most frequently selected features correlated with the code length, yet no single feature dominated. Top-ranked features typically accounted for 11–60% of positive SAGE mass, with the remainder distributed across opcode frequencies, AST node counts, and graph structure metrics.

Discussion. While intuitive signals such as code length and input size appear among top features, the bulk of predictive power is fragmented across dozens of low-level indicators (opcode distributions, AST motifs) without a unifying pattern. Furthermore, feature rankings vary substantially across models: UniXcoder prioritizes lexical cues (e.g. token_count), whereas GPT-OSS-120B emphasizes opcodes and graph density. This heterogeneity, coupled with only modest AUROC scores, indicates that classical software metrics correlate only weakly with LLM comprehension. If they do, the correlation is largely driven by code length, i.e., a modest negative correlation between program length and model success. The data supports that LLM capability on this task transcends traditional human-centric complexity measures.

4.3 Non-Human Signal via a Shadow Model

Having shown that human-designed metrics weakly predict performance, we investigate whether *non-human* signals can do better.

We fine-tune UniXcoder [3] as a feature-free shadow model to predict a target model’s correctness on each (p, x, y) triple from raw inputs, bypassing handcrafted features and probing for predictive cues beyond human intuition.

Experiment Setup. For each target model, we train a dedicated shadow model on an 80/20 stratified train/test split of the corresponding success labels. The input is a concatenated sequence $\langle p, x, y \rangle$ tokenized. Following OCEAN [7], we train for a single epoch and report performance on the test-set.

Results. Shadow models achieve mean AUROC of 0.859. Individual performance varies by target model: GPT-OSS 120B (AUROC 0.834), Mistral Small 24B (AUROC 0.872), Llama 3.3 70B (AUROC 0.849), UniXcoder (AUROC 0.878), and Phi-4 (AUROC 0.864).

Discussion. The shadow model’s substantial improvement over human metrics (mean AUROC 0.859 vs. 0.634) confirms that non-human signals exist and are learnable. Unlike the human-metric classifier, which struggles to integrate weak indicators, the shadow model directly encodes high dimensional interactions between code structure, I/O context, and target-model behavior. However, the moderate AUROC (~ 0.86) reveals limits: even feature-free learned representations leave uncertainty. Those limits suggest that LLM failure patterns contain both systematic components (captured by the shadow model) and context-dependent factors beyond what raw code and I/O reveal. The consistent AUROC across diverse target models, indicates these signals generalize within the shadow modeling framework, yet they remain model-specific. Moreover, learned representations remain opaque: while shadow models outperform human metrics, they offer no interpretable explanation of *why* certain samples fail. Taken together, these results demonstrate that LLM code comprehension patterns transcend classical complexity measures but reflect model-specific, partially predictable regularities rather than deterministic or universal principles.

5 Related Work

The number of benchmarks for code understanding is increasing: SWE-bench tasks models with resolving real GitHub issues across multi-file contexts [4]. CRUXEval assesses execution reasoning via input-output prediction on short Python programs [2]. In contrast, we do not assess the quality of generated code. Instead, we evaluate binary model classifications over given programs, thereby shifting from coarse aggregate scores to capability-level diagnostics.

A second line of work leverages software engineering metrics to assess LLM-generated code. Sepidband *et al.* study whether human-centric metrics can proxy the correctness of generated code [10]. We, however, do not evaluate the quality of generated code. Instead, we compute human-centric metrics on the input program and use them to assess whether the model’s output classification is correct.

Complementary to our approach, Kim *et al* [5] propose Lachesis, a framework that predicts whether large language model reasoning will yield a correct outcome by analyzing the structural properties of sampled reasoning paths. Using graph- and sequence-based representations, Lachesis anticipates inference success. While our shadow models likewise predict model outcomes, Lachesis targets reasoning-level reliability, whereas we focus on input-level code comprehension.

Closest to our length-based finding, Mächtle *et al.* show that increasing input length degrades LLM performance on security-related tasks [6], and Rando *et al.* report similar degradation on their novel code-comprehension benchmark [9]. However, both analyses focus solely on length and omit other static and structural complexity indicators. Our results both corroborate this length effect and systematically broaden the metric space, while further introducing a feature-free shadow predictor that captures model-specific regularities beyond human-designed metrics.

6 Conclusion and Outlook

We introduce a diagnostic framework for assessing code comprehension in large language models that (i) reframes evaluation as a binary I/O-consistency judgment, (ii) relates performance to human-centric software metrics, and (iii) trains feature-free shadow predictors directly from raw code and I/O context. We found that classical measures, such as size, AST/CFG structure, or bytecode statistics, correlate only weakly with LLM success (AUROC 0.63). Shadow models trained on raw inputs achieve moderately higher predictive performance (AUROC 0.86), indicating that learned representations capture systematic patterns beyond handcrafted features.

These findings motivate three implications: First, aggregate benchmark scores obscure specialization: model selection should be task- and instance-aware rather than guided solely by headline accuracy. Second, the weak alignment between human metrics and LLM behavior underscores the opacity of current systems and motivates interpretability methods that move beyond handcrafted features. Third, the moderate shadow model performance (AUROC 0.86) highlights inherent unpredictability in LLM code comprehension, cautioning against over-reliance on predictive models in safety-critical applications.

Future work must develop hybrid approaches that combine human-interpretable diagnostics with learned predictors while acknowledging irreducible uncertainty in LLM behavior.

Acknowledgments

This work has been supported by funding from the Agentur für Innovation in der Cybersicherheit GmbH (Cyberagentur, project SOVEREIGN)

References

- [1] Ian Covert, Scott M. Lundberg, and Su-In Lee. Understanding global feature contributions with additive importance measures. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual*, 2020.
- [2] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.
- [3] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unix-coder: Unified cross-modal pre-training for code representation. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2022, Dublin, Ireland, May 22–27, 2022, pages 7212–7225. Association for Computational Linguistics, 2022.
- [4] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [5] Naryeong Kim, Sungmin Kang, Gabin An, and Shin Yoo. Lachesis: Predicting LLM inference accuracy using structural properties of reasoning paths. In *IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning, DeepTest@ICSE 2025, Ottawa, ON, Canada, May 3, 2025*, pages 17–20. IEEE, 2025.

- [6] Felix Mächtle, Nils Loose, Tim Schulz, Florian Sieck, Jan-Niclas Serr, Ralf Möller, and Thomas Eisenbarth. Trace gadgets: Minimizing code context for machine learning-based vulnerability prediction. *CoRR*, abs/2504.13676, 2025.
- [7] Felix Mächtle, Jan-Niclas Serr, Nils Loose, Jonas Sander, and Thomas Eisenbarth. OCEAN: open-world contrastive authorship identification. In *Applied Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23-26, 2025, Proceedings, Part II*, volume 15826 of *Lecture Notes in Computer Science*, pages 459–486. Springer, 2025.
- [8] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir R. Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale AI for code dataset for learning a diversity of coding tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021.
- [9] Stefano Rando, Luca Romani, Alessio Sampieri, Yuta Kyuragi, Luca Franco, Fabio Galasso, Tatsunori Hashimoto, and John Yang. Longcodebench: Evaluating coding llms at 1m context windows. *CoRR*, abs/2505.07897, 2025.
- [10] Melika Sepidband, Hamed Taherkhani, Song Wang, and Hadi Hemmati. Enhancing llm-based code generation with complexity metrics: A feedback-driven approach. In *49th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2025, Toronto, ON, Canada, July 8-11, 2025*, pages 1416–1426. IEEE, 2025.